Contents

Martelage — the Evidence Package standard 2
What an evidence package is 2
Why the standard exists: the honest sentence 2
The five mechanisms, and exactly what each buys 3
What it is not 4
The name 5
Status 5
Martelage — using the standard 6
Verify: the common case 6
Verify without any tooling 8
Produce: what a second project must build 8
Chain and anchor: the two layers that look outside the package 9
The anchor slot: three named profiles 10
What you should not expect 14
Where the evidence lives 15

Martelage — the Evidence Package standard

Draft 0.1. A directory format for records, plus one manifest that states the package's scope, its place in a package chain, and whether anyone other than the producer holds its digest.

- How to use it — verifying and producing a package (usage.html)
- The specification — `spec/MARTELAGE-0.1.md`
- The repository README — `README.md`

What an evidence package is

An evidence package is a **directory**, not an archive (§4.1 MA-16). It holds:

- **Record stream files.** JSON Lines files whose records are hash-chained within a stream: each record carries `seq`, which runs 1..n with no gaps, and `prev`, the `body_sha` of the record before it (§6, §7.1–7.2). Two writers' records are never merged into one stream (§6.6).
- **One manifest**, `package.json`, listing every other file in the package with its `sha256` and its byte length, and stating three things beyond that file list:
  - the package's **scope** — what it contains and, separately, what it does *not*, each absence carrying a meaning word from a fixed vocabulary (§8, MA-54 to MA-60);
  - its **place in a package chain** — `prev.package_sha` and `prev.how`, saying whether a predecessor exists and where that value came from (§7.4);
  - its **anchor statement** — whether a party other than the producer holds this package's digest, and under which named profile (§9). The slot has named profiles at this version, and one of them is an eIDAS qualified seal — the anchor profiles (usage.html#anchors) (§9.3).

The **package digest** is `sha256` over the exact bytes of the manifest. The manifest carries no digest of itself; because it lists every other file's digest, that one value transitively pins every file in the package, including the scope statement and the anchor claim (§7.3.1 MA-47).

All of it is checkable offline, with a POSIX shell and `sha256sum`, by someone who has never seen the producer: the zero-tooling check on the usage page (usage.html#zero-tooling) is a normative property of the format, not an example (§4.6 MA-21).

Why the standard exists: the honest sentence

The threat model comes first, because every mechanism in the standard is derived from it (§3). **The adversary is the operator.** The party that produced the package is assumed to have administrative control of the machine it was produced on, and can therefore rewrite the system's records, its auxiliary state, the exported files, the manifest and the producing binaries — recomputing every digest inside the package correctly and

consistently (§3.1 MA-10). Nothing inside a package proves anything against that party acting coherently; the standard calls this not a defect to be designed away but the residual every mechanism is measured against (§3.2 MA-11).

So the standard states its own residual, and every conforming validator MUST print the following two sentences verbatim on every run *that names a package* — including a run that reports the package conformant, and including a run that ends at exit code 2 because the package could not be read (§3.3 MA-12):

> A package proves that what it contains was not altered after it left; it cannot prove that what it contains is all there was, and against an operator who rewrote everything before the first anchor it proves nothing.
> A coherent tamper performed before the first observation is invisible to every mechanism in this standard.

That is the whole position. A format that names its adversary can be argued with; a format that does not cannot (§3).

MEASURED on this tree: the reference validator prints both sentences, word for word, in a `residual` block on every run that names a package — the usage page pastes the block from a real run (usage.html#verify).

The five mechanisms, and exactly what each buys

Each mechanism is specified and reported separately, because their powers differ by orders of magnitude and a reader will otherwise credit the weakest with the strength of the strongest (§3.4). This is the standard's own table:

The five mechanisms, each with its exact power (§3.4)

#	Mechanism	What it buys	What it does not
i	Internal consistency (§5.1–5.2, §6.2, §7.1–7.2)	A partial or sloppy rewrite shows as a break: a moved body digest at a fixed position, a gap, a count that disagrees with the payload.	A wholesale, internally consistent rewrite passes.
ii	Staleness against the previous package (§7.4)	Converts an invisible omission into a break visible one package later — if someone kept the earlier package.	An operator who starts a fresh chain and was watched by nobody.
iii	Witnesses outside the audited subject (§3.6)	Forces the operator to rewrite *more* places consistently.	Stops the operator. Each extra rewrite is another place to contradict a copy someone else holds — it multiplies cost, never prevention.
iv	The enrollment gap (§3.5)	Nothing. It is named so it is not hoped away.	—
v	The external anchor (§9)	Proves a digest existed at a time, held by someone other than the operator.	Truth, completeness, or that the operator told the witness everything.

The standard also gives the one-line reading of its own layers, which is the sentence to carry away (§7.6 MA-53):

> Layers 0–2 stop tampering *with a package you hold*; layer 3 bounds tampering *across packages a verifier held*; only layer 4 binds the package to a party outside the operator's reach.

Two consequences are stated as requirements rather than left to inference. A validator report MUST show which layers were armed, and MUST NOT let an unarmed layer read as a pass (§7, §10.4 item 6). And a chain proves order and integrity over what is *present*: it does not prove completeness — a consistently-chained prefix, and a consistently-chained subset, pass every check in layers 0 and 1 (§7.2.3 MA-45).

What it is not

- **Not a record format.** It does not say what an auditable system should record, in what schema, or with what semantics. It defines the outer layer that carries someone else's records; the record grammar is the payload's own business, named and not specified here (§0.5 MA-5).
- **Not a claim about the records.** A conformant package may contain records that are false, and may omit records that existed. Conformance says the package is well-formed and self-consistent; it says nothing about truth or completeness (§2.2 MA-7).
- **Not a certification, and not a registry.** No registration, no permission and no fee is required. A third party claims conformance by publishing a statement in its own name, naming the requirements it relies on and any it does not satisfy, with the reason; the standard certifies no one (§2.4 MA-9).
- **Not a service.** The reference validator runs offline with a POSIX shell and the utilities §10.1 names: no network, no interpreter beyond those, no key material, no service, no producer (§10.1 MA-68).
- **No keys, and no third party required for the base tier.** A package with no third party, no seal, no timestamp and no key material is fully conformant; nothing in the conformance definition requires any of them (§9.1(a) and (b), MA-61). The anchor is an optional slot, not a tier.
- **Not a ladder.** There is one level, deliberately: a ladder reads as "the lower tier is deficient", which is backwards, since the unanchored tier is the adoption surface (§2.3 MA-8).
- **Not a promise about the producer.** A validator is not required to know a producer; `producer` is a claim under the producer's own name (§10.8 MA-75).

One implementation is named in the standard's Annex A, and only there. That annex is informative: nothing in it is required to conform, and removing it would leave the standard whole.

The name

Martelage is the forestry marking of trees with a hammer, by a party other than the owner: a durable mark, attributable to whoever struck it, that decides which trees are retained. The standard's own text does not gloss its name anywhere (MEASURED: no occurrence of "forestry" in `spec/MARTELAGE-0.1.md` or `README.md`); the sentence above is the name's plain sense, an INTERPRETATION, and no clause of the standard depends on it.

Status

- **Draft 0.1** — published for adoption and criticism; not yet a stable standard. A change to this document's requirements may be breaking while it is a draft; a change to the published wire format (`"standard":"martelage","v":1`) must not be (§11.6 MA-81).
- **Every normative requirement carries an identifier**, `MA-1` … `MA-82`, so a conformance report can cite what it reports as met or unmet; a clause that cannot be cited cannot be tested (§0.3 MA-3). MEASURED: all 82 identifiers appear in the specification.
- **Licence split:** the specification text is CC BY 4.0; the reference validator and all code in the repository are Apache-2.0 — see `LICENSE`.
- **Where it lives:** the specification is `spec/MARTELAGE-0.1.md`; the reference validator is `validator/martelage-verify`; the forty-seven test vectors are under `vectors/`, each with an `expected.txt`.

Martelage — using the standard

Draft 0.1. Verifying a package you were handed; producing one from your own records; and what a validator must admit it cannot check.

- What an evidence package is, and what it does not prove (index.html)
- The specification — `spec/MARTELAGE-0.1.md`
- The repository README — `README.md`

Every command on this page was run by us on this tree (HEAD `90dbceb`) and its output pasted verbatim. Blocks marked MEASURED are copied from those runs; a statement that is a reading of the specification rather than a run is marked INTERPRETATION, in the manner the standard itself requires (§0.4 MA-4).

Verify: the common case

You have a package directory. You run the reference validator against it. That is the whole procedure — there is nothing to install, no account, no key, no network:

```
sh validator/martelage-verify vectors/01-minimal-conformant/pkg
```

MEASURED, exit code `0`. This is the run's standard output in full, unedited; nothing was written to standard error:

```
martelage-verify — MARTELAGE-0.1 reference validator

package      vectors/01-minimal-conformant/pkg
digest       667229f663fa3d0c79d738ba0d016c99e10ff05dd794d86a01edd3f8df0a5c1a   (sha256 of package.json: this is the
↳ package digest)
standard     martelage v1 (kind package-manifest)
subject      c419edf957fda5f89c2988273d3d00e9e5b2a69ebe7f6bca22644da52bc129f3 (kind note-log)
producer     example / by-hand 0 / contract martelage-example-notes 1
created_at   1700000200
prev         null (how: none)
files        2
records      2 over 1 stream(s)

scope
  contains:
    - 2 record(s) on stream(s): note
  not_contains:
    - anything not listed above [not-recorded] this package is a hand-made example for the standard

files
  OK   records.jsonl               abf920fbb49003e1b06badc085e74f8396713bca64d2f123eccdb256e6a36bad  776 bytes
↳ role=records
  OK   notes.md                    798dab2402394de24954a4e5d0b8c806d8185ac2e54a95ec3ef22e1aeeba807f  55 bytes
↳ role=document

streams
  FILE                STREAM          N     HEAD          VERDICT
  records.jsonl       note            2     6e28c56d1a81  OK

anchor
  NOT ANCHORED — this package's digest is held by no one but its producer

layers
  layer 0  body digests       checked
  layer 1  stream continuity  checked
  layer 2  package digest     667229f663fa3d0c79d738ba0d016c99e10ff05dd794d86a01edd3f8df0a5c1a
  layer 3  package chain      not armed (no --prev-package given) — omission across packages is NOT checked
  layer 4  anchor             not claimed by this package
```

```
refusals
  none

findings
  none

ignored
  2 field(s) this validator does not know, counted per (object, field) — ignored, never guessed at (§5.2.11 MA-33,
↳ §11.5 MA-80):
  'records.jsonl' line 1     generator
  'records.jsonl' line 1     counts

not decidable
  nothing — every check this validator implements could reach a verdict on this package (MA-70)

residual
  A package proves that what it contains was not altered after it left; it cannot prove that what it contains is all
↳ there was, and against an operator who rewrote everything before the first anchor it proves nothing.
  A coherent tamper performed before the first observation is invisible to every mechanism in this standard.

verdict      CONFORMANT
```

Read the block in this order and the report tells you what it did and did not do. The `layers` block names each layer and whether this run armed it. The `anchor` block says, in words, that no one but the producer holds this digest. The `residual` block is the §3.3 sentence, printed on every run that names a package. The `ignored` and `not decidable` blocks are sections of their own, so that a field the validator did not understand and a check it could not reach are never invisible (§10.4 items 9 and 10, MA-71).

A refused package reads the same way, with the failing check named — MEASURED on `vectors/03-stream-gap/pkg`, an interior record removed, exit code 1. Trimmed here to the refusal and the verdict; the rest of that report has the shape shown above:

```
refusals
  payload: records.jsonl stream 'note': seq 4 where 3 was expected;

verdict      NON-CONFORMANT
```

Exit codes

The exit code is the verdict; the tail of the output is not. The four codes are the standard's (§10.3 MA-70, and `README.md`):

The exit codes (§10.3)

code	meaning
`0`	**CONFORMANT** — every check the run could perform passed. An unanchored package exits 0; the NOT-ANCHORED line has been printed.
1	**NON-CONFORMANT** — a check failed: a digest, a length, a gap, a broken predecessor link, a torn line, a manifest count, an unknown scope word, an unsupported version, a chain break against `--prev-package`, a claimed anchor that is malformed or that the evidence contradicts.
2	**USAGE OR UNREADABLE** — the invocation is wrong, or an input could not be read (an unreadable file is not an absence).
3	**CANNOT DECIDE** — an input the validator needs to decide is absent: anchor evidence for a claimed anchor, trust anchors for P3, or a profile-specific verifier this validator is not. The validator MUST name the missing input.

Why code 3 exists. An instrument that cannot return "undecided" returns something else instead. A claimed anchor that cannot be checked is neither conformant nor refuted, and reporting it as either would be a false statement; it is not conformant enough to pass, so 3 is non-zero (§10.3). This is not a nicety — it is the same discipline as the residual: the format says what it could not establish.

Verify without any tooling

The manifest is canonical and lists every file with its digest, so a verifier with nothing but a POSIX shell can check the file set without any validator. This one-liner is a normative property of the format: a manifest whose canonical form does not admit it is non-conformant (§4.6 MA-21).

```
grep -o '{"path":"[^"]*","sha256":"[0-9a-f]*"' package.json |
  sed 's/{"path":"\([^"]*\)","sha256":"\([0-9a-f]*\)"/\2  \1/' > SHA256SUMS
sha256sum -c SHA256SUMS
```

MEASURED, run from inside a copy of `vectors/01-minimal-conformant/pkg` made in a temporary directory — the one-liner writes `SHA256SUMS` into the current directory, so do not run it inside the repository unless you mean to add that file. The three lines below are the written `SHA256SUMS` followed by `sha256sum -c`, exit code `0`:

```
abf920fbb49003e1b06badc085e74f8396713bca64d2f123eccdb256e6a36bad  records.jsonl
798dab2402394de24954a4e5d0b8c806d8185ac2e54a95ec3ef22e1aeeba807f  notes.md
records.jsonl: OK
notes.md: OK
```

What that check covers is the file set: every listed file present, and each file's bytes equal to the digest the manifest states. What it does not cover is everything above the file list — the record envelopes, the per-stream sequence and predecessor chain, the manifest counts, the scope vocabulary and the anchor claim (INTERPRETATION, from §4.4 and §7). For those you need the validator.

Produce: what a second project must build

There is no library and no service. Production is decentralised and cheap; checking is centralised and shared — an adopter does not reimplement the validator, it runs the published one, or reads it and writes its own against the specification (Annex F). A project that already writes dated records to durable files needs exactly two things:

1. **An emitter** that puts its records into the envelope of §6.2: canonicalise each body, compute `body_sha`, assign `stream` and `seq`, wire `prev`, print the stream manifest first. For a project whose records are already one JSON object per line,

Annex F prices this at roughly 80–120 lines in any language.

2. **A packager** that lists its files with digests, writes a canonical manifest with an honest scope, a `prev` from a one-line state file, and `anchored:false` — Annex F prices it at roughly 60–100 lines.

Those two line counts are the standard's own estimate (Annex F), not a measurement of any particular project; treat them as a PROJECTION in the §0.4 sense.

Annex E is a worked package written out by hand with `printf` and `sha256sum`: two files, one shell, no network, no keys, no service. It is the whole of what a second project must produce to start, and a stranger can copy it. `README.md` records that executing that example verbatim, in an empty directory, with nothing from the repository on the path except the validator, produces a package the validator reports CONFORMANT (exit 0) — that is the README's measurement; this page did not re-run it.

Do not expect a validator to live inside a package. A package is not required to carry one, and a validator found inside a package proves nothing about that package, because it sits inside the directory the operator can rewrite; checkability comes from the published validator or from the verifier's own implementation, obtained through a channel the operator does not control (§4.5 MA-20).

Chain and anchor: the two layers that look outside the package

Two verifier inputs arm the two layers that reach beyond the package directory (§10.2 MA-69). Neither is required for the base tier — and an unarmed layer must never read as a pass (§7, §10.4 item 6).

`--prev-package DIR` — arms layer 3

Without it, a record dropped between two packages is invisible: the omission happened outside the package you hold, and nothing inside this package records that it ever existed. MEASURED, both arms on `vectors/11-chain-omission` — first the successor alone, exit code `0`, with the layer reported as unarmed. Trimmed to the `layers` block and the verdict:

```
layers
  layer 0  body digests        checked
  layer 1  stream continuity   checked
  layer 2  package digest     20be1945067e73fbffebabd55d2408d1241c74db7a934304bd49b83d6c72ecf7
  layer 3  package chain       not armed (no --prev-package given) — omission across packages is NOT checked
  layer 4  anchor              not claimed by this package

verdict      CONFORMANT
```

Now the same package with its predecessor supplied — the same omission, exit code `1`. Trimmed to the refusal and the verdict:

```
sh validator/martelage-verify --prev-package vectors/11-chain-omission/pkg-1 vectors/11-chain-omission/pkg-2
```

```
refusals
  chain: stream 'note' is not a prefix of the same stream in the supplied previous package — records were dropped,
↳ re-ordered or rewritten across packages (§7.4.2 MA-50)

verdict      NON-CONFORMANT
```

That is the whole power of the layer, and it is bounded exactly as §7.4.3 says: it converts a silent omission into a break visible one package later — *if someone kept the earlier package* — and it does not stop an operator who starts a fresh chain and is watched by nobody (§7.4.3 MA-51).

`--anchor-evidence HEX|FILE` — arms the anchor

"Anchored" is a property of (package, verifier's evidence), not of the package alone: the same package is verified-anchored for a verifier who brought the holder's digest, and cannot be decided for one who did not (§9.5 MA-66). MEASURED on `vectors/04-anchor-unverifiable`, a package claiming a p1 anchor with no evidence supplied — exit code `3`. Trimmed to the `anchor` block, the `not decidable` block and the verdict:

```
anchor
  CLAIMED — profile 'p1-second-party-digest', binds 'manifest-digest'
  claim: holder 'witness@example-archive', channel 'ask witness@example-archive for the digest of this package'

not decidable
  anchor: profile p1 is claimed but --anchor-evidence was not supplied — the holder's digest is the input this check
↳ needs (§10.3 MA-70)

verdict      CANNOT DECIDE — 1 input(s) this validator needs were absent
```

And the same claim with the holder's digest supplied (`vectors/05-anchored-verified/evidence.txt`) — exit code `0`. Trimmed to the `anchor` block and the verdict:

```
sh validator/martelage-verify --anchor-evidence vectors/05-anchored-verified/evidence.txt
↳ vectors/05-anchored-verified/pkg
```

```
anchor
  ANCHORED — profile 'p1-second-party-digest', binds 'manifest-digest'
  claim: holder 'witness@example-archive', channel 'ask witness@example-archive for the digest of this package'
  evidence: the holder's digest equals this package's digest — VERIFIED (§9.3 MA-64)

verdict      CONFORMANT
```

Which profile a package names decides what a verifier can do with its anchor, and one of the three is an EU qualified seal. The profiles have their own section.

The anchor slot: three named profiles

The anchor is a slot with named profiles, not one vendor: §9.3 names three at this version and leaves the slot open to others (§9.1(c) MA-61). It gets a section of its own because layer 4 is the only layer whose witness the operator cannot recompute (§7.5 MA-52) — and because the profile a package names is the thing that decides what a verifier can actually check. What no profile changes is the claim's force: an anchor never says the record is

true or complete, only that a digest existed, at a time, by someone else (§3.7 MA-15, §7.5 MA-52).

The three anchor profiles at this version (§9.3)

profile	wire string — the value of `anchor.profile`	what the base validator does with it
P1	`p1-second-party-digest`	Verifies it fully: the holder's digest must equal the package digest (§9.3 MA-64).
P2	`p2-append-only-log`	Compares a digest the log states to the package digest. A digest that contradicts the package is a refusal (exit 1); a digest that matches still leaves inclusion undecided, so it reports exit 3 and names what it could and could not check (§9.3, §10.3 MA-70).
P3	`p3-eidas-qualified-seal-asic-e`	Cannot verify a signature, so it reports exit 3 and names the profile-specific verifier it is not (§9.3, §10.3 MA-70).

`P1`, `P2` and `P3` are labels for those strings, never values: a manifest writing `P3`, or the profile's name in any other spelling, is refused as an unknown profile (§9.2 MA-63, §9.4 MA-65). The wire string is what a validator matches.

`p3-eidas-qualified-seal-asic-e` — the EU qualified seal

P3 is the profile for a deployment that already has to answer to EU law. In the standard's own terms it is an ETSI EN 319 162-1 ASiC-E container holding the package files with a CAdES or XAdES signature over their digests, created by a legal person *other than the operator*, with a qualified electronic time stamp and optional long-term validation (§9.3 MA-64). The party who seals is the point: a seal is only an anchor because someone the operator is not made it.

The claim is a reference, not the seal. A manifest cannot carry a value derived from its own digest (§9.1a MA-62), so the container and the trust anchors stay on the verifier's side and the claim only says where to get them. This is the claim shape, verbatim from §9.3:

```
{"container":…,"sealer":…,"seal":"qualified-electronic-seal",
 "timestamp":"qualified"|"none","ltv":<bool>}
```

What the reference validator does with that claim is nothing, and it says so. It cannot verify a signature — that needs a signature verifier and trust anchors — so it reports exit code `3` CANNOT DECIDE and names the profile-specific verifier requirement (§9.3, §10.3 MA-70). That is the standard's own rule applied rather than an omission: a claimed anchor the verifier cannot verify MUST NOT pass (§9.1(e) MA-61). Nor does a specialised verifier get a lighter contract for being specialised — a profile-specific verifier MUST report under §10, the same exit codes and the same report items (§9.3, §10.4 MA-71) — so "we sealed it" is not a way to skip the report.

Status: built, by a second implementation. The base validator still stops at exit code `3` and still names the profile-specific verifier it is not — that has not changed and is not going to,

because §9.1(b) keeps keys and trust anchors out of the base tier. The verifier it names is `validator/p3-verify`: a second implementation that shares no code with `martelage-verify`, runs under `python3` and calls the `openssl` binary for every cryptographic operation. It reads no CRL, contacts no OCSP responder, and has no network path at all.

```
validator/p3-verify \
  --container vectors/34-seal-verified/seal.asice \
  --trust-anchors vectors/pki/anchors \
  --operator-cert vectors/pki/operator.crt.pem \
  vectors/34-seal-verified/pkg                    # exit 0
```

It takes the container and the anchors on the verifier's side, as §10.2 (MA-69) says a P3 input must be, plus one input of its own: `--operator-cert`, the operator's certificate. Without that certificate the "legal person *other than the operator*" clause cannot be checked, so the run reports `3` for that clause and says the input is missing. It never assumes the signer is someone else — which is why no P3 run reaches exit `0` without it.

The clause is checked over the whole signer set. A signature entry may carry more than one signer, and the operator only has to be one of them. The verifier enumerates *every* signer of *every* entry from openssl's `-signer` output, holds that count against the signerInfos the entry itself states, refuses the container if the two disagree, and compares every one of them with `--operator-cert`: if any signer is the operator, the run refuses and names it. The report prints the signer count and every signer's subject, window and public-key digest — never a single subject standing in for "the signer". The same set is what the certificate-date check runs over: every signer, every certificate in the chain each signer reaches, and the timestamping authority's own chain, all held to the time the verified token establishes and never to this host's clock.

What `p3-verify` checks, and the four exit codes (§10.3 MA-70)

code	meaning for P3
`0`	**CONFORMANT** — the container is well-formed (no duplicate entry name, every covered path inside the package), the CAdES signature verifies over the ASiC manifest, *every* signer of *every* signature entry chains to a supplied anchor and none of them is a CA, every sealed digest is the package's, the RFC 3161 token verified, every certificate the run established — every signer, the whole chain, the TSA's own chain — was valid at the claimed time, and no signer's key is the operator's.
`1`	**NON-CONFORMANT** — a malformed container (a wrong `mimetype`, a duplicate ZIP entry name, a covered path that leaves the package, an entry the manifest lists and the ZIP does not hold, a sealed digest that no longer matches the file on disk), a signature that does not verify, a signer certificate that is a CA, any signer or chain certificate that was not valid at the claimed time, a token from a timestamping authority that was not, a seal made with the operator's own key *or co-signed with it*, or `--require-timestamp` given with no token.
`2`	**USAGE OR UNREADABLE** — a bad invocation, an unreadable input, or a package that does not claim P3 at all (that package is the base validator's).
`3`	**CANNOT DECIDE** — an input was not supplied: the container, trust anchors, `--operator-cert`, or a verified timestamp to establish the claimed time. A signature that verifies but reaches no supplied anchor is *this* code, not a refusal: the verifier was not given the anchor that decides it (§9.5 MA-66).

What it still cannot check, on every run. The verifier prints a section headed `not checkable by this verifier (every run)` before its verdict — and prints it to stderr on an exit-2 run as well, so "on every run" is true of the runs that have no report — and the list below is that section's content: **revocation** — no CRL is read and no OCSP responder is contacted, so "valid at the claimed time" means the certificate dates and never revocation status; **long-term validation** — LTV material is reported present or absent and never validated; the **truth and completeness** of the records, which is limit (iii) below and is kept distinct from the residual of §3.3; the **signer's legal identity** beyond the subject its certificate claims; whether the certificate was **issued for sealing** at all (this standard defines no key-usage profile for a seal, so the chain is verified with `-purpose any` and that is said rather than assumed — the `basicConstraints` CA bit is the one key-usage-shaped property that *is* checked, because a CA is not a sealing party); and a **re-forged history**, limit (ii). A run that could not check something never reads as having checked it, and when no verified time exists the date question is reported CANNOT DECIDE rather than skipped.

What it is not. Not a qualified trust service, not an eIDAS certification, and not a claim of compliance with ETSI EN 319 162-1. The container shape it verifies is the manifest-signature form — a detached CAdES over `META-INF/manifest.xml`, carrying the covered files' digests in that manifest — and the tool says so in its own report, because the EN 319 162-1 multi-file concatenated-digest form is not what it parses and it will not report that as verified. It does not verify XAdES. It does not make the base validator verify seals, and it does not change the base tier's tool list (§10.1 MA-68).

The fixtures are a **throwaway** test PKI under `vectors/pki/` — two roots, an intermediate, a second intermediate with the same key and subject that expired in 2021, a sealer, a second sealer under the same intermediate, a sealer under the withheld root, an expired sealer, the operator, a timestamping authority and a second TSA certificate with the same key and subject that expired in 2021 — with its private keys committed on purpose, because a vector that cannot be rebuilt from what is in the tree is not reproducible. The dates are fixed with `-not_before`/`-not_after`, so the expired-signer, expired-intermediate and expired-TSA vectors are expired on any host at any time; the keys are regenerated on each run, so `sh tools/mkpki.sh && sh tools/mkp3.sh` rebuilds the PKI and the containers together. Both run offline. Fourteen vectors (34–47) cover this profile, each with an arm that passes and an arm that fails, differing by one thing: a digest that moved, an anchor that is withheld, an operator that sealed, an operator that *co-signed*, a CA used as the sealing party, an intermediate or a TSA that expired, a ZIP entry name that appears twice, and a covered path that leaves the package.

§9.3 requires three limits to be stated wherever P3 is offered. They are the profile's terms, not its small print:

- **(i) A seal made with the operator's own key is worthless against the operator**, because they hold the key. That is why the verifier compares *every* signer of *every* signature entry: a co-signer does not launder a self-seal, and "the first signer was someone else" is not the clause being checked. Only a seal by a different legal person, or a qualified time stamp, is out of their reach — and even then the trust service attests only what it was handed (§9.3).
- **(ii) A single seal does not stop a re-forged history.** The operator can doctor a package and have *that* sealed. What defeats it is custody of prior packages by the other party, so the seal multiplies the chain and does not replace it (§9.3).
- **(iii) The legal presumption that attaches to a qualified seal is a presumption about integrity and origin, not a technical guarantee of truth or completeness**, and it must be kept distinct from the residual of §3.3 in any document that cites both (§9.3).

What P3 rests on is in Annex B, and nothing beyond it is claimed here. Item 10 adopts eIDAS (Regulation (EU) No 910/2014) and ETSI EN 319 162-1 as the basis of P3, with the three limits above, and records why the presumption stops where it does: it attaches to the integrity and origin of the sealed data, not to its truth or completeness. Item 11 records eIDAS 2.0 (Regulation (EU) 2024/1183) as introducing the EU Digital Identity Wallet and qualified attestations of attributes about *persons* — and marks the consequence a PROJECTION in the §0.4 sense (MA-4): no change for a package at this version, because the seal, timestamp and preservation instruments P3 uses are the 910/2014 ones (Annex B items 10–11).

What you should not expect

A validator that did not say what it cannot check would be an advertisement. Every validator must admit the following (§10.7 MA-74):

- whether the record is **true**;
- whether the package is **complete** — all that ever existed;
- anything that happened **before the first observation or the first anchor** (§3.5);
- the **producer's binaries**;
- the **honesty of a witness**, including a same-host witness (§3.6);
- **where a witness runs**, or whether it is outside the audited subject's reach: the package can say so in `bound[]` or in its scope, and that statement is the producer's claim, not a checked fact;
- whether a `subject.id` is a **digest** rather than a hostname or an account name: a validator refuses the path shapes it can see and cannot distinguish the rest (§5.2.3);

- **who may read the records**: a payload's own readability classification is a classification by the publisher, and this standard enforces nothing about it — a validator only echoes it.

The same limits seen from the package's side — what a package is not, and the residual every validator prints — are on the first page (index.html#why). In one line: a package proves what it contains was not altered after it left; it cannot prove it is all there was, and against an operator who rewrote everything before the first anchor it proves nothing.

Where the evidence lives

The repository ships forty-seven hand-made packages under `vectors/`, each with an `expected.txt` recording the run, the expected exit code, the literal lines it must print, and the `MA-` identifiers the case exercises (MEASURED: 33 directories, 33 `expected.txt` files). None of them is exported from a producer system; they were constructed for the standard (`README.md`).

A standard whose only validator has never rejected anything is a description of its author's output. So the claim that the validator can fail is itself measured: take a copy of `validator/martelage-verify`, delete the one line that compares a record's `prev` to its predecessor's `body_sha` (line 828 of the file as shipped), leave everything else alone — `sh -n` on the mutant is clean — and run the vector built for exactly that check. MEASURED on this tree, both runs trimmed to the refusal block and the verdict:

```
# as shipped
$ sh validator/martelage-verify vectors/02-broken-predecessor/pkg
refusals
  payload: records.jsonl stream 'note': prev at seq 2 does not name seq 1;

verdict      NON-CONFORMANT
exit=1

# mutant: line 821 deleted
$ sh /tmp/noprev vectors/02-broken-predecessor/pkg
refusals
  none

verdict      CONFORMANT
exit=0
```

The mutant still reports `0` on `vectors/01-minimal-conformant` — the mutation removed that check and nothing else (MEASURED). Vector 02 was built for exactly this: its sequence numbers, counts, head and every digest are consistent, so the `prev` comparison is the only check that can catch it. `README.md` lists one such mutant for each check added by review; a check whose removal changes nothing is not load-bearing.

Specification: `spec/MARTELAGE-0.1.md`. Repository and licence: `README.md`, `LICENSE`.

Raising a defect: open one against the *sentence*, not the code — quote the `MA-` identifier and the text, and say what it lets a producer or a verifier claim that the format cannot deliver (see `README.md`, "Raising a defect").